

Purchase Order Forecasting for a Single OEM

Documenting Everything...

GitHub Repository : [Click here](#)

June 15, 2026

1 Model Training and Testing on Dummy Dataset

Goal: To forecast purchase orders (demand) for individual car models and variants at various dealerships up to 12 months ahead for a single automotive OEM.

Data: An LLM-generated six dummy dataset mimicking real-world dealership sales, inventory, and vehicle lifecycles.

Primary Metrics:

- **WAPE** (Weighted Absolute Percentage Error): Lower is better.
- **Accuracy** (100% - WAPE): Higher is better.

2 Training, Testing, and Feature Engineering

2.1 How Training and Testing was Performed

To prevent the model from "Data Leakage", I used **Time-Series Holdout Validation**. I shifted the target variable backward by the forecast horizon (e.g., 1 to 12 months). Then, I trained the models on all historical data up to a specific cutoff date and tested their predictions strictly on the **unseen final 12 months** of the dataset to evaluate real-world performance.

2.2 Feature Engineering

Before feeding data to the models, I engineered several features to help the algorithms understand the patterns:

- **Autoregressive Time Lags (1, 2, 3, 6, 12 months):** I created columns showing exactly what the sales were in previous months to capture short-term momentum and annual seasonality (like Diwali spikes).
- **Rolling Averages (3 and 6 months):** I calculated the moving average of sales over recent windows to smooth out random, noisy spikes and reveal the true underlying trend.

- **Months Since Launch (Lifecycle Tracking):** I calculated exactly how old the car design was in months. This explicitly tells the model where the car is in its lifecycle, preventing it from predicting peak sales for a 4-year-old vehicle.
- **True Demand Reconstruction:** Instead of using raw sales (which drop if there is a supply shortage), I calculated "True Demand" by combining retail deliveries with order backlogs and inventory stockouts.

3 Model Evolution & Experiments (File name: Final_model.py)

3.1 The Starting Point: Baseline CatBoost Model

What I thought: I needed a strong baseline. Since automotive data has a lot of categorical features (like dealer state and fuel type), I decided to use a single, massive CatBoost Regressor.

What I did: I fed all the historical data into the CatBoost model to predict 1, 3, 6, and 12 months ahead for all dealerships at once.

The Outcome:

- **1-Month Horizon:** 30.3% WAPE (**69.7% Accuracy**)
- **3 to 6-Month Horizons:** 32.0% WAPE (**68.0% Accuracy**)

The Problem: The model was too "monolithic." Asking CatBoost to train on the entire dataset for every single horizon was computationally exhausting. The pipeline kept timing out or crashing during the 3-month forecasting stage because it was simply too heavy to run. Accuracy was also not good.

3.2 Trying to Fix the Zeros

What I thought: Car sales data has many months where a dealership sells exactly zero units of a specific car. Standard regression models get confused by too many zeros. I researched and found that a two-step "Hurdle Model" could fix this.

What I did: I split the model into two steps:

1. **Classifier:** First, predict IF a car will sell at all (Yes/No).
2. **Regressor:** If Yes, use an ensemble of models to predict HOW MANY will sell.

I also added a "Hierarchical Reconciliation" step to make sure the individual dealer predictions added up to the total regional predictions.

The Outcome:

- **1-Month Horizon:** 30.7% WAPE (**69.3% Accuracy**)
- **3 to 6-Month Horizons:** 34.1% WAPE (**65.9% Accuracy**)

The Problem: The accuracy actually dropped slightly. More importantly, the main issue remained: the pipeline was still crashing. Training a classifier *plus* multiple regressors for every single row of data was even slower than the first iteration.

3.3 Dealer Clustering + Stacking Ensemble

What I thought: I realized I needed to break the massive problem into smaller chunks. Dealerships in big cities behave differently than rural ones. If I cluster them, I can train smaller, faster, and more specialized models. Furthermore, relying on just one algorithm leaves blind spots, so a Stacking Ensemble would allow different algorithms to cover each other's weaknesses.

What I did:

- I used K-Means to group dealerships into distinct clusters based on their historical sales behavior.
- Instead of running one massive model, I ran a loop to train a **Stacking Ensemble** for each cluster independently.
- **The Algorithms & Their Purpose:** I built a heterogeneous stack utilizing three distinct gradient boosting frameworks as "Base Learners," followed by a linear "Meta-Learner":
 - **CatBoost:** Included primarily for its powerful native handling of categorical features (dealer state, car model, fuel type) without needing aggressive one-hot encoding. Its symmetric tree structure also makes it highly resistant to overfitting on low-volume, sparse data.
 - **LightGBM:** Included for its extreme speed and leaf-wise tree growth. It excels at finding complex splits in heavy numerical features, making it perfect for extracting patterns from my autoregressive lags and rolling averages.
 - **XGBoost:** Included as a highly robust, traditional depth-wise gradient booster to capture any deep non-linear relationships that LightGBM or CatBoost might have missed.
 - **Ridge Regression:** I used Ridge (an L2 regularized linear regression) to blend the predictions coming from the three base models. Ridge acts as a stabilizer—it mathematically learns which base model is most trustworthy for different scenarios while penalizing extreme outlier predictions, preventing the final ensemble from overfitting.

The Outcome:

- **1-Month Horizon:** 17.4% WAPE (**82.6% Accuracy**)
- **3 to 12-Month Horizons:** 29.8% WAPE (**70.2% Accuracy**)

The Result: Accuracy increased drastically. The pipeline finally ran from start to finish without crashing. Breaking it down by clusters fixed the speed issue, and designing a purposeful Stacking Ensemble pushed the 1-month accuracy past our target, achieving over 80% accuracy.

Resources:

- **Dataset (Google Drive):** Dummy Dataset
- **Codebase (Google Colab):** Python script for dummy

4 Model for Tata Motors Dataset

Goal: To forecast purchase orders (true demand) for individual car models and highly specific variants up to 12 months in advance for a single automotive OEM (Tata Motors).

Primary Metrics:

- **WAPE** (Weighted Absolute Percentage Error) – *Lower is better.*
- **Accuracy** (100% - WAPE) – *Higher is better. Target: > 80%.*

Important Links (Final Files):

- **Script for cleaned_car_sales dataset:** `kaggle_model.py`
- **Script for tata_enterprise dataset:** `tkkn3.py`
- **Raw Dataset:** `tata_motors_datas`
- **Cleaned Dataset:** `cleaned_car_sales.csv`
- **Final Dataset:** `tata_motors_enterprise_demand.csv`

5 TATA Motors Dataset

What I thought: To build an accurate model, I needed to start with real OEM data and slowly refine it as the models became more advanced. Later, I realized that sales data will not be enough for forecasting the demand and hence forecasting the purchase order because it only records how many cars were actually delivered, not how many people *wanted* to buy them (due to part shortages, canceled after test drives, etc.).

What I did: My data evolved in three distinct stages:

1. **Raw Data (`tata_motors_datas`):** The initial messy dataset.
2. **Cleaned Data (`cleaned_car_sales.csv`):** I cleaned the raw data by removing old, discontinued cars and formatting the columns. This was used for all Phase 1 experiments.
3. **Enterprise Data (`tata_motors_enterprise_demand.csv`):** I added mathematically correlated dummy features and to the cleaned data. I shifted the target from *Sales Units* to *Net Bookings* (actual demand), and added leading indicators like *Web Traffic Views*, *Wait Times*, and *Average Discounts* etc.

6 Model training and testing for cleaned_car_dataset

Now I began to train and test new model for cleaned TATA Motors Data.

1. The Strategy (Python File Name: `Kaggle_model.py`)

For this cleaned tata dataset, I implemented a per-model standardisation step: scaling the target variable by its historical maximum ($y_{scaled} = \frac{y}{y_{max}}$). This uniform feature space allows the LightGBM model to learn pure behavioral demand signatures independently of raw volume magnitudes.

2. The Machine Learning Stack

I used a Global LightGBM Regressor with an $L2$ regression objective, which aligns perfectly with the scaled targets. To capture recent market dynamics, I applied exponential recency weights ($\lambda = 0.05$) to penalize older data. Additionally, I clustered the vehicles using K-Means ($k = 4$) based on their statistical profiles. This cluster assignment acts as a powerful categorical feature, guiding tree splits for "high-volume stable" versus "low-volume volatile" models.

3. Hierarchical Reconciliation

Real-world forecasting requires granular "bottom-up" variant predictions to perfectly match the highly stable "top-down" brand total. Because individual variant forecasts are noisy, their raw sum rarely equals the total brand prediction. I solved this using a **Weighted Least Squares (WLS)** reconciliation based on inverse variance. Models with high historical variance are adjusted more to close the gap, while highly confident predictions (low variance) remain largely untouched.

4. Bias Correction and Evaluation

I implemented a per-model multiplicative bias correction learned from cross-validation residuals. After inverse-transforming predictions back to raw purchase order units, my final Reconciled WAPE settled at 34.6%. (that is nearly 65% accuracy)

1. Model Training and Testing on the Tata Enterprise Dataset (Python File: tkkn3.py)

To make the dataset more realistic, I added several useful features generated using an LLM. During initial testing, the model achieved nearly 90% accuracy, but this was later found to be caused by data leakage. After fixing the issue, the performance dropped to a more realistic level.

2. Algorithm used

The dataset is highly sparse, with around 70–80% of variant-wise monthly sales being zero. To handle this, I trained 12 separate LightGBM models using the Tweedie objective function, which works well for data with many zero values.

3. Evaluation

After applying proper walk-forward cross-validation, the final error was 85.33%, indicating that the model struggled to accurately predict future demand.

4. Conclusion

The main challenge was the lack of sufficient and informative data. The dataset contains many zero-sales records and mostly historical sales information, which was not enough to accurately forecast demand for the next 12 months. Additional factors such as market trends, seasonality, and promotions would likely improve performance.

For a good dataset to predict the demand and purchase order , i need features:

- New Gross Bookings (Orders)
- Order Cancellations
- Net Bookings
- Retail Deliveries (Registrations)
- Website Configurations
- Test Drive Requests
- Dealership Inquiries
- Effective Transaction Price
- Consumer Offers / Cash Discounts
- Financing Rates (Auto Loan EMI)
- Marketing Spend (Ad-Stock)
- Months Since Launch (Variant)
- Months Since Facelift
- Internal Cannibalization Flag
- Variant Status
- Fuel Prices
- EV Charging Parity
- Stock Market Index (Nifty Auto / Nifty 50)
- Festive Season Intensity
- Average Wait Time (Lead Time)
- Dealer Inventory Days
- Production Constraint Flags